

AWS Lambda Integration with Amazon SQS

To Begin with the Lab

Summary of the Lab

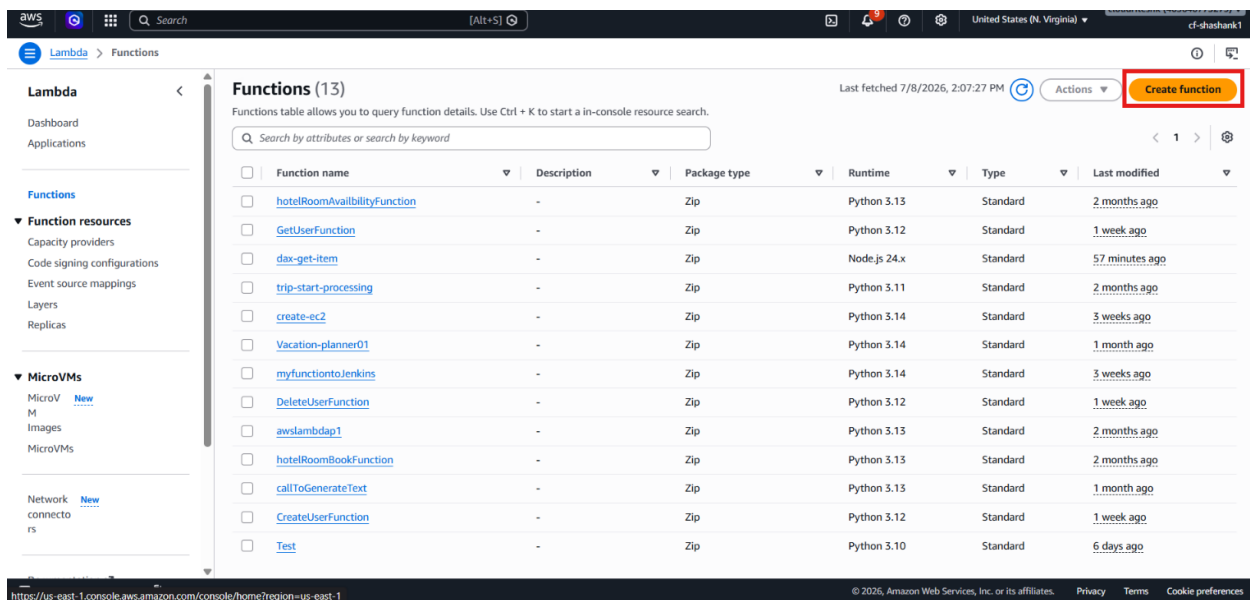
In this lab, we create a Lambda function using the Node.js runtime and configure Amazon SQS as its event source (trigger). Whenever a message arrives in the queue, AWS Lambda is automatically invoked to process it.

To demonstrate DLQ behavior, the Lambda function intentionally fails every invocation. This causes Amazon SQS to retry message delivery according to the queue's **Maximum Receive Count**. Once the retry limit is exceeded, the message is automatically moved to the configured Dead Letter Queue.

Lab Steps

Step 1 – Create the Lambda Function

- Open the **AWS Lambda** console.
- Click **Create Function**.



- Choose: Author from scratch
- Provide the following details:
- Function Name: HelloLambda
- Runtime: Node.js 20.x
- Architecture: x86_64

Create function Info

Choose one of the following options to create your function.

☒ **Author from scratch** Start with a sample or create a new function from scratch. World example.

☐ **Use a blueprint** Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image** Select a container image to deploy for your function.

Basic information

Function name Info
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

Custom settings Info

Durable execution - new Build resilient, stateful applications with automatic failure recovery.

EC2 capacity provider - new Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

- Execution Role:
- Create a new role with basic Lambda permissions
- Click **Create Function**.
- The Lambda function is now ready.

Step 2 – Attach Amazon SQS as the Trigger

- Inside the **HelloLambda** function, click **Add Trigger**

HelloLambda

Successfully created the function "HelloLambda". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

Function overview Info

Diagram | **Template**

Layers (0)

Function ARN
arn:aws:lambda:us-east-1:463646775279:function:HelloLambda

Description
-

Last modified
1.3 seconds ago

+ Add trigger | **+ Add destination**

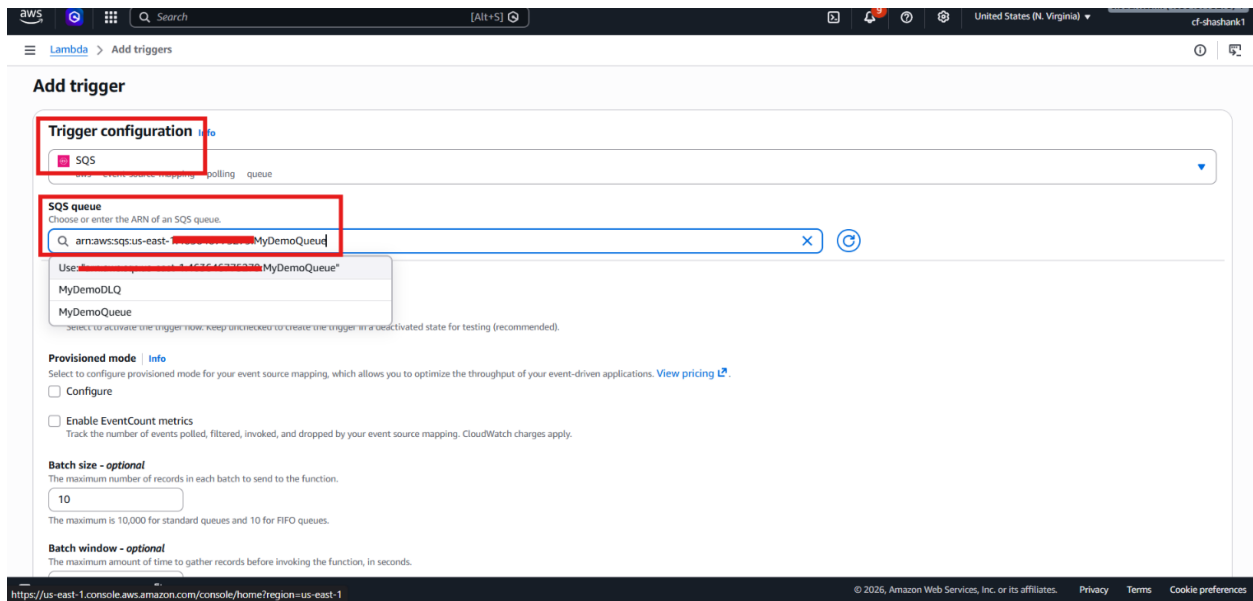
Code source Info

Open in Visual Studio Code | **Update**

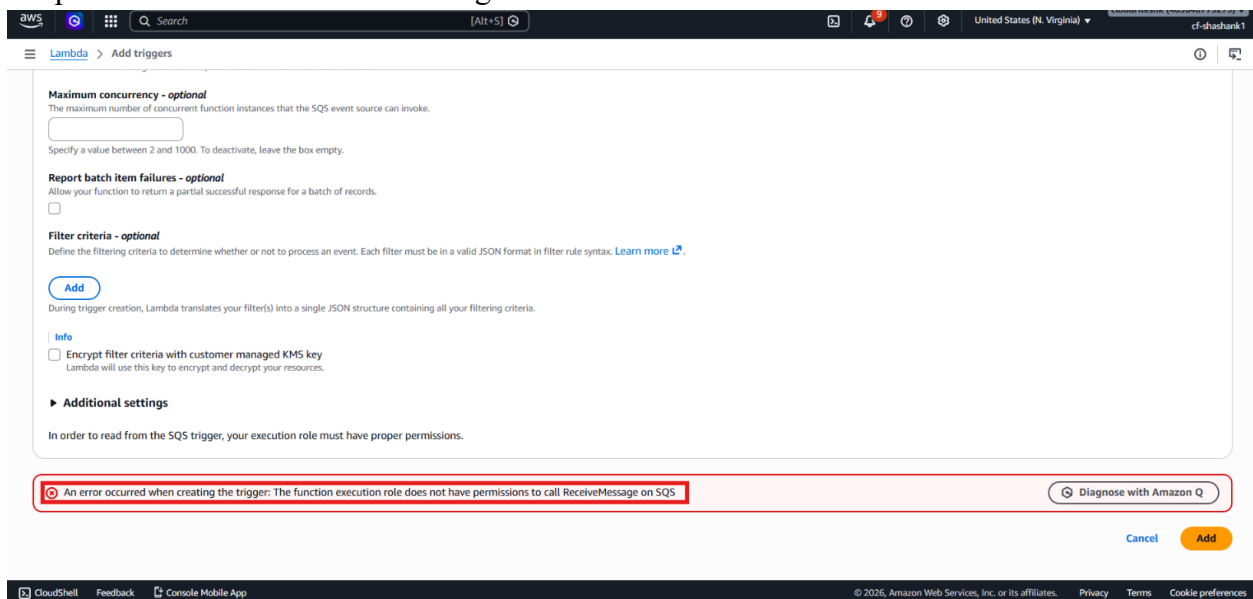
EXPLORER

- HELLOLAMBDA
 - index.mjs

- Select source as **SQS**.
- Choose your existing queue: **MyDemoQueue**
- Leave all remaining settings as default and click **Add**.



- After this step, you will get an error saying unable to create trigger as it does not have the permission to call RecieveMessages



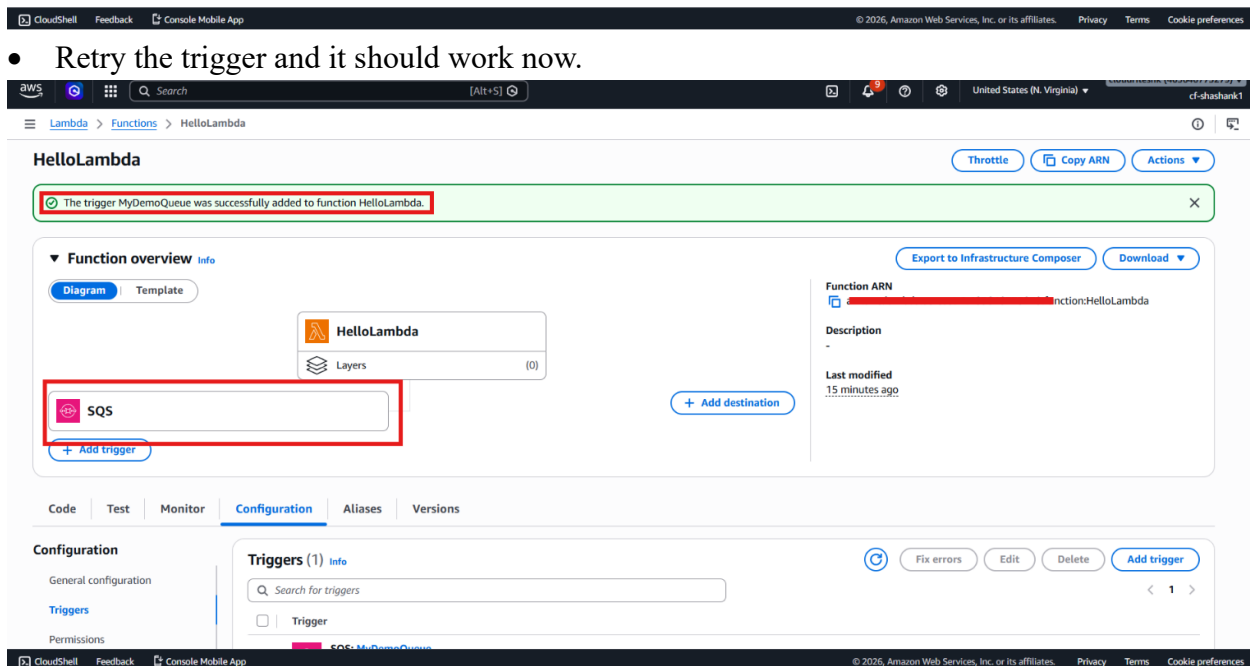
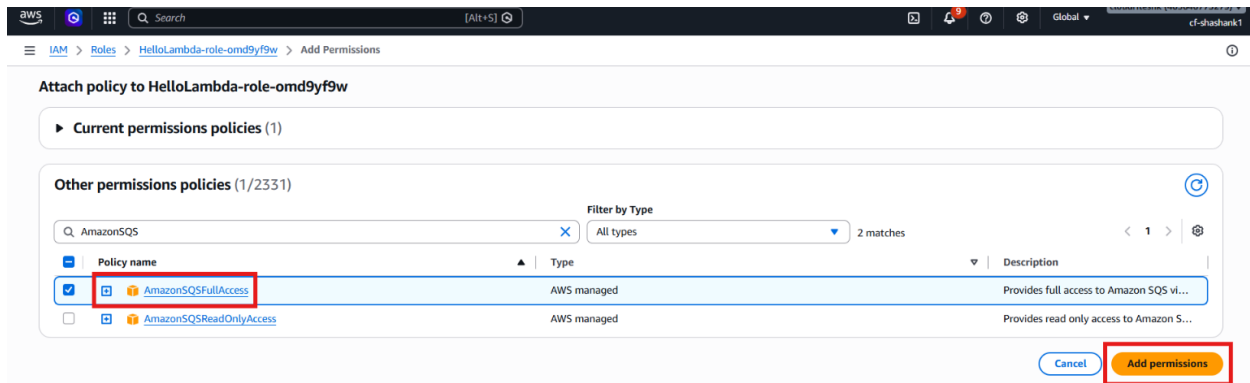
- To fix this, we must add another permission to the role.
- Navigate to **HelloLambda Function** → **Configuration** → **Permissions** → **Role Name**.

The screenshot shows the AWS Lambda console for a function named 'HelloLambda'. The 'Configuration' tab is selected, and the 'Permissions' sub-tab is highlighted. The 'Execution role' section shows the role name 'HelloLambda-role-omd9yf9w'. The 'Resource summary' section shows 'Amazon CloudWatch Logs' with 3 actions and 2 resources. The 'Function overview' section shows the function ARN 'arn:aws:lambda:us-east-1:463646775279:function:HelloLambda' and the last modified time '10 minutes ago'.

- Now open the Role and Click on **Add Permission → Attach Policy**.

The screenshot shows the AWS IAM console for the role 'HelloLambda-role-omd9yf9w'. The 'Permissions' tab is selected, and the 'Add permissions' button is highlighted. The 'Permissions policies' section shows a table with one policy: 'AWSLambdaBasicExecutionRole-6dc8e0ce-8e5a-4ff...'. The 'Permissions boundary' section is not set. The 'Generate policy based on CloudTrail events' section is also visible.

- Add **AmazonSQSFullAccess** and Click **Save**.

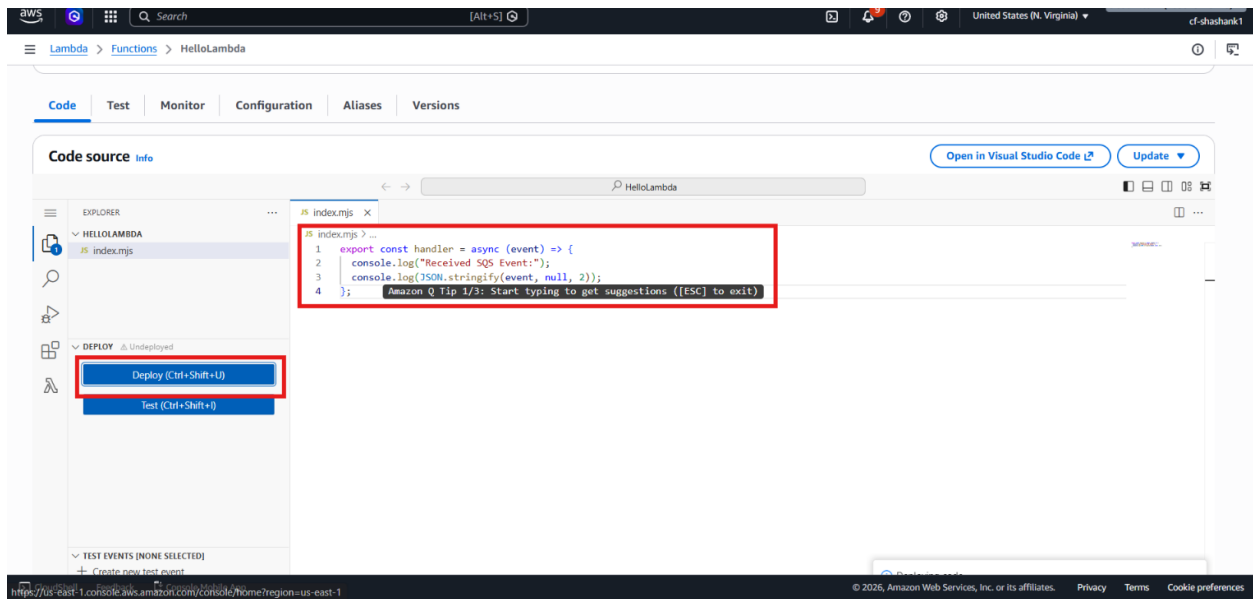


Step 3 – Replace the Lambda Code

- Open the default **index.mjs** file.
- Replace the existing code with:

```
export const handler = async (event) => {
  console.log("Received SQS Event:");
  console.log(JSON.stringify(event, null, 2));
};
```

- Click **Deploy**.



Code Explanation

- Logs the complete SQS event received by Lambda.
- Intentionally throws an exception.
- Simulates a processing failure for DLQ testing.
- Causes Amazon SQS to retry the message until the maximum receive count is reached.

Verification

- **HelloLambda** function is created successfully.
- **SimpleQueue** is attached as the Lambda trigger.
- Lambda code is deployed successfully.
- The execution role contains the **AWSLambdaSQSQueueExecutionRole** policy (if not automatically attached).
- Lambda is ready to process messages arriving in the SQS queue.